Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco
Department of Computer Science, University of Nevada, Las Vegas
Faculty Research Mentor: James Andro-Vasko, Ph.D.
Department of Computer Science, University of Nevada, Las Vegas

Summer Undergraduate Research Fellowship 2026
Office of Undergraduate Research, UNLV

Code, handouts, test script, raw results:
github.com/seancnc003/UNLVDockerIDEs
hub.docker.com/r/seancnc/unlv-cpp-ide
hub.docker.com/r/seancnc/unlv-x86-ide

Introduction

Setting up software can delay the start of a programming course. Several hundred UNLV students a year in introductory C++ (CS 135) and x86-64 assembly (CS 218) must install four programming tools on their own laptops before coursework begins. Some newer Macs cannot run the assembly course's existing virtual machine, a simulated computer with a 19 GB disk. Other universities report the same problem [3, 6].

Purpose/Aim

I packaged the course tools and a code editor into a container that runs on the student's own computer and opens in a browser tab.

Question. On four tested configurations, can the same package run assignments, preserve coursework after deletion, and support debugging?

Measured. Download size and time, editor-opening time, memory use, and build time.

Contribution. Earlier work describes packaged programming workspaces [1, 3, 4]. This project evaluates UNLV's assembly-course package on four configurations and documents its ARM debugger limit.

Methods

Test systems. Three rented virtual computers and my own laptop provided four test configurations (Table 1).

Same test on each. One automated checklist, unchanged on every machine:

1 Download → open the editor → build and run four assignments

2 Test the debugger → delete the container → check the saved coursework

TECHNICAL DETAILS

Machines. AWS m8i.large (Intel) and m8g.large (ARM Graviton); Azure D4s_v5 (Intel, Windows 11 Pro); all rented, under $10 in total. 2021 MacBook Pro M1 Pro, owned. This one command, typed by the student, starts the workspace and connects it to the coursework folder:

```
docker run -p 127.0.0.1:8218:8080 -v ~/UNLV/x86-workspace:/home/coder/workspace
seancnc/unlv-x86-ide
```

Checks. The same download on every machine, confirmed by a file identifier (checksum). All four assignments must build and run without error; the parallel-work assignment (several streams of work at once) must also give its expected answer. The debugger must show the paused program's values and position.

Timing. Editor opening time runs from starting the container until the editor's web page answers the browser. It excludes installing Docker (not measured) and the download (reported separately). Timed on the first start and after replacing the container; second-start results are not shown here.

Memory and builds. Memory is the container's own use (editor and tools; not the browser or Docker), idle and during assignments. Each assignment was built once.

Results

All four tested computers built and ran four assembly-course assignments. Neither ARM computer, including the Apple Silicon Mac, could use the debugger.

A debugger pauses a program so a student can inspect it. ARM is the processor family used in Apple Silicon Macs.

4 of 4 computers
built and ran all four assignments

Editor opening
Intel: 0.5–0.6 s
ARM: 4.8–6.7 s

552 MB download
Docker installation is additional

Debugger available
Intel: 2 of 2
ARM: 0 of 2

Only first-start editor times are shown. Docker installation was not timed; download times are reported separately.

Table 1. Capabilities and measurements for the four tested configurations. Times describe these systems only. Memory in MiB.

Measure	Windows 11 PC Intel, 4 cores, 16 GB rented (Azure)	Linux PC Intel, 2 cores, 8 GB rented (AWS)	Apple Mac M1 Pro (ARM), 16 GB owned MacBook Pro	Linux on ARM Graviton ARM, 2 cores, 8 GB rented (AWS)
All four assignments built and ran	yes	yes	yes	yes
Parallel-work answer correct	yes	yes	yes	yes
Coursework kept after deletion	yes	yes	yes	yes
Debugger available	yes	yes	no	no
Package download time (s)	53	16	45	13
Editor opens after first start (s)	0.6	0.5	4.8	6.7
Memory while idle (MiB)	56	54	263	255
Peak memory during assignments (MiB)	90	not captured*	358	334
Slowest assignment build (s)	0.6	0.3	3.1	3.6
Checks met / failed / unavailable	13 / 0 / 0	13 / 0 / 0	12 / 0 / 1	12 / 0 / 1

*The Linux PC finished faster than the memory sampler could record a peak. The computers differed in provider and resources; times describe these test systems. "Unavailable" is the debugger check, recorded rather than passed or failed on ARM.

Figures

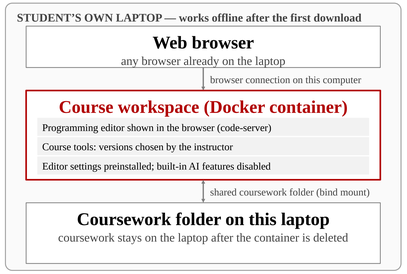
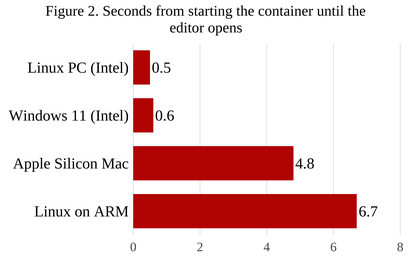


Figure 1. Everything runs on the student's own laptop. The internet is needed only for the first download.



First-start times after Docker is installed and the package downloaded. ARM machines run the Intel-built package through a translator.

Key Terms

Container a ready-made package of a course's tools. Docker is the free software that runs it.

Virtual machine a whole simulated computer inside a computer; the course's current one is a 19 GB disk, a different measure from a download.

IDE / editor the application students write programs in; here it opens in a browser tab.

Assembly programming with instructions tied closely to the processor.

Build convert a written program into a form the computer can run.

Debugger a tool that pauses a running program so a student can look inside it. Some assignments require one.

Working memory temporary space used while software runs, separate from saved files.

Processor families a processor carries out program instructions. Intel-style and ARM processors understand different basic instructions; Apple Silicon Macs use ARM.

Translator software that runs instructions written for a different processor family. Some features may be unavailable.

Discussion

Why debugging matters. Some assignments require students to pause a program and inspect its values. On the tested ARM computers those steps still need an Intel computer or the campus lab; everything else ran.

Possible explanation. The package uses Intel-style instructions because that is what the course teaches. ARM computers run it through translation software, which may explain the debugger limit. Other hardware and software differences were not isolated.

Conclusions/Future Research

Conclusion. These four tests support assignment execution and coursework preservation, and show a debugging barrier on the two ARM computers.

Not tested. Student setup success, time saved, or help requests. Four machines are examples, not a survey.

Next. Seek university ethics (IRB) approval for a classroom study in both courses [3, 4, 6]; evaluate a revised ARM container that could enable debugger use on ARM.

References/Acknowledgements

[1] D. J. Malan. Containerizing CS50. ITiCSE 2024. [3] S. Valstar, W. G. Griswold, L. Porter. Using DevContainers to Standardize Student Development Environments. ITiCSE 2020. [4] K. Fernalld et al. Lightweight Symphony: Reducing CS Student Anxiety with Standardized Docker Environments. SIGITE 2023. [6] D. P. Harvie et al. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019. [8] H. Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 15(19), 2025. [12] UNLV Dept. of Computer Science. Student Center: Remote Access and File Storage. tux.cs.unlv.edu, accessed Aug. 2026.

Supported by the UNLV Office of Undergraduate Research Summer Undergraduate Research Fellowship. Every number on this poster comes from the published test script's output files, archived in the project repository.